Visão e texto – consulta rápida

1 · Redes convolucionais

Módulo N3_06
imagens: (N, C, A, L)

01 IMAGENS COMO TENSORES

N3_06 · §1

`torch.tensor(imgs).unsqueeze(1)`	(N, A, L) → (N, 1, A, L): acrescentar o canal.
`imgs / 16` `imgs / 255`	Normalizar as intensidades para 0–1.
Canais	1 em tons de cinzento; 3 a cores (RGB); numa camada interna, um por filtro.

02 CONVOLUÇÃO

N3_06 · §2

`for i in range(A - k + 1):`
`  for j in range(L - k + 1):`
`  s[i, j] = (img[i:i+k, j:j+k]`
`  * filtro).sum()`
	Um filtro desliza pela imagem; em cada posição, soma os produtos. Resultado: um mapa de ativação.

`[[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]`	Filtro de Sobel: arestas verticais. Numa CNN, os filtros são aprendidos.
`nn.Conv2d(1, 16, kernel_size=3,` `  padding=1, stride=1)`	16 filtros 3×3: 16·9 + 16 = 160 parâmetros, para qualquer tamanho de imagem.
`(entrada + 2p − k) // s + 1`	Tamanho da saída. k = 3 e p = 1 mantêm o tamanho; s = 2 reduz para metade.

03 POOLING

N3_06 · §3

`nn.MaxPool2d(2)`	Máximo de cada quadrado 2×2: metade da altura e da largura.
`a.reshape(A//2, 2, L//2, 2)` `.max(axis=(1, 3))`	O mesmo em NumPy.
Para quê	Menos cálculo e tolerância a pequenas deslocações.

04 UMA CNN

N3_06 · §4

`nn.Sequential(`
`  nn.Conv2d(1, 16, 3, padding=1),`
`  nn.ReLU(), nn.MaxPool2d(2),  # 8 → 4`
`  nn.Conv2d(16, 32, 3, padding=1),`
`  nn.ReLU(), nn.MaxPool2d(2),  # 4 → 2`
`  nn.Flatten(),`
`  nn.Linear(32 * 2 * 2, 10))`
	Extrator de atributos (convolução → ReLU → pooling) + classificador. As entradas do Linear têm de bater com o que sai do extrator.

`for c in modelo:` `  x = c(x); print(x.shape)`	Acompanhar a forma camada a camada.
`nn.CrossEntropyLoss()(logits, y)`	10 classes: 10 *logits*; y inteiros (`torch.long`).
`torch.softmax(logits, dim=1)` `logits.argmax(dim=1)`	Probabilidades (somam 1) · classe prevista.

05 AVALIAR E INTERPRETAR

N3_06 · §5–6

`ConfusionMatrixDisplay(` `  confusion_matrix(y, prev)).plot()`	Que classes se confundem.
`modelo[0].weight  # (16, 1, 3, 3)`	Filtros da 1.ª camada: vários são detetores de arestas.

06 ROBUSTEZ E DATA AUGMENTATION

N3_06 · §7

Rede densa	Regras por píxel: falha quando a imagem se move.
CNN	Reconhece o padrão em qualquer posição; tolera pequenas deslocações.
Data augmentation	Variações aleatórias no `__getitem__` (nova em cada época).
torchvision	`transforms.RandomAffine(degrees=10, translate=(0.1, 0.1))`.

Dígitos 8×8 deslocados 1 píxel: rede densa 42 %, CNN 63 %, CNN com *data augmentation* 94 % (N3_06 §7).

Visão e texto – consulta rápida

2 · Texto e linguagem natural

07 PREPARAR O TEXTO

N3_07 · §2

`re.findall(r"\w+", texto.lower())`	Tokenizar: palavras em minúsculas, sem pontuação.
`Counter(tokens).most_common(10)`	Explorar as palavras mais frequentes.
`[t for t in tokens if t not in STOP]`	Retirar *stopwords* (artigos, preposições…).
Cuidado com o «não»	Nunca o tirar em análise de sentimento: «não é bom» viraria «bom».
Testar cada passo	Stopwords, *stemming*, lematização: podem ajudar ou prejudicar.

08 BAG-OF-WORDS E TF-IDF

N3_07 · §3–4

`CountVectorizer(token_pattern=r"\w+")`	Contagens: uma coluna por palavra. Perde a ordem.
`ngram_range=(1, 3)`	Também pares e trios de palavras: «não é bom».
`stop_words=lista  min_df=5`	Retirar palavras · ignorar termos raros.
`TfidfVectorizer(...)`	Pesa pela raridade: idf = ln((1+n)/(1+df)) + 1, depois norma L2.
`X.nnz / np.prod(X.shape)`	Matriz esparsa: quase tudo é zero.

09 CLASSIFICAR

N3_07 · §5

`make_pipeline(TfidfVectorizer(...), LogisticRegression(C=10))`	O vocabulário aprende-se só com o treino.
`pd.Series(m[-1].coef_[0], index= m[0].get_feature_names_out())`	Pesos: + puxa para positiva, – para negativa.
`m.predict_proba(["O preço não é mau."])`	Palavras fora do vocabulário são ignoradas.

10 EMBEDDINGS

N3_07 · §6

`nn.Embedding(n_palavras, 16)`	Tabela: um vetor denso por palavra, aprendido por retropropagação.
`nn.EmbeddingBag(n, 16, mode="mean")` `modelo(ids, inicios)`	Média dos vetores de cada texto (como o fastText); ignora a ordem.
`u @ v / (norm(u) * norm(v))`	Semelhança do cosseno: palavras usadas da mesma forma ficam próximas.
Pré-treinados	word2vec, GloVe, fastText: relações gerais («rei – homem + mulher ≈ rainha»).

11 TRANSFORMERS

N3_07 · §7

Atenção	O vetor de cada palavra depende de todas as outras do texto: «boa» em «não foi boa».
Tokens de subpalavras	Palavras raras partidas em pedaços conhecidos.

```
from transformers import pipeline
clf = pipeline("sentiment-analysis",
               model=NOME_DO_MODELO)
clf(["A entrega não foi boa."])
```

Modelo pré-treinado da Hugging Face (precisa de Internet na 1.ª vez).

Na prática	Afinar (*fine-tuning*) um modelo pré-treinado, ou usar um LLM com boas instruções.

! LIÇÕES DO MÓDULO

Pré-processamento	Stopwords sem o «não»: de 89 % para 96 %, mais do que trocar de modelo.
Bigramas não chegam	«não é bom» dá «é bom»: só com trigramas (ou stopwords) a negação aparece inteira.
Começar simples	TF-IDF + regressão logística: rápido, interpretável, muitas vezes suficiente.

Ordem de evolução: contar palavras → pesar pela raridade → vetores densos aprendidos → atenção ao contexto.